```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
builder.Services.AddScoped<IMessageSender, EmailSender>();
builder.Services.AddScoped<IMessageSender, SmsSender>();
builder.Services.AddScoped<IMessageSender, FacebookSender>();
```

Then you can inject `IEnumerable<IMessageSender>` into `RegisterUser`, as shown in listing 9.8. The container injects an array of `IMessageSender` containing one of each of the implementations you have registered, in the same order as you registered them. Then you can use a standard `foreach` loop and call `SendMessage` on each implementation.

Listing 9.8 Injecting multiple implementations of a service into an endpoint

```
string RegisterUser(
    string username,
    IEnumerable<IMessageSender> senders)              ❶
{
    foreach(var sender in senders)                    ❷
    {                                                 ❷
        Sender.SendMessage($”Hello {username}!”);     ❷
    }                                                 ❷

    return $"Welcome message sent to {username}";
}
```

❶ Requests an IEnumerable injects an array of IMessageSender

❷ Each IMessageSender in the IEnumerable is a different implementation.

WARNING You must use `IEnumerable<T>` as the handler parameter type to inject all the registered types of a service, `T`. Even though this parameter will be injected as a `T[]` array, you can’t use `T[]` or `ICollection<T>` as your constructor argument. Doing so will cause an `InvalidOperationException`, similar to that in figure 9.1.